Programowanie obiektowe

Wykład 5

Dziedziczenie

Dziedziczenie w programowaniu obiektowym

Jeden z czterech filarów programowania obiektowego (obok abstrakcji, hermetyzacji i polimorfizmu).

> **Dziedziczenie** (ang. *inheritance*) to mechanizm, który pozwala tworzyć nowe klasy na podstawie już istniejących.

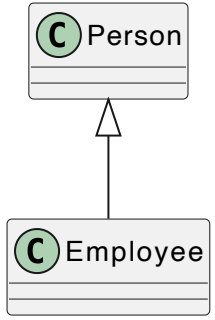
- Dziedziczenie pozwala nowej klasie przejąć właściwości i zachowania klasy istniejącej.
- Dziedziczenie umożliwia ponowne wykorzystanie kodu i tworzenie hierarchii typów.
- Dziedziczenie tworzy szczególną (**bardziej specyficzną**) odmianę klasy istniejącej.

Przykłady:

- `PrzesyłkaPolecona` jest rodzajem `PrzesyłkiPocztowej`
- `Pracownik` jest rodzajem `Osoby`
- `Koło` jest rodzajem `Figury`

Dziedziczenie w UML

- W UML dziedziczenie jest reprezentowane przez **relację generalizacji**.
- Symbolem jest linia ciągła zakończona **pustym trójkątem** skierowanym w stronę **klasy bazowej**.
- Przeważnie klasa bazowa znajduje się na górze, a klasa pochodna niżej.

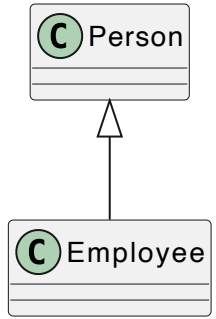


- **Klasa bazowa (nadrzędna, *super class*)** – klasa, z której dziedziczymy.
- **Klasa pochodna (podrzędna, *sub class*)** – klasa, która dziedziczy po innej klasie.

Dziedziczenie w C++

- Dziedziczenie deklarujemy w **klasie pochodnej** – to klasa pochodna wskazuje na swoją klasę bazową.

```
// Klasa bazowa:
class Person {
    // ...
};

// Klasa pochodna:
class Employee : public Person {
    // ...
};
```

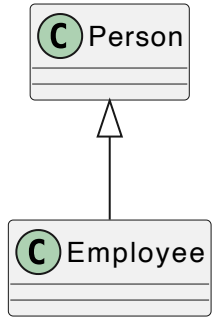


- Dziedziczenie oznaczamy symbolem `:`.
- Dziedziczenie wymaga określenia poziomu dostępu (tutaj: `public`).

Dziedziczenie w Javie

- Dziedziczenie deklarujemy w **klasie pochodnej** – to klasa pochodna wskazuje na swoją klasę bazową.

```
// Klasa bazowa:
public class Person {
    // ...
}

// Klasa pochodna:
public class Employee extends Person {
    // ...
}
```

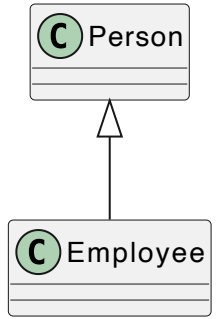


- Dziedziczenie oznaczamy słowem kluczowym `extends`.

Dziedziczenie w C#

- Dziedziczenie deklarujemy w **klasie pochodnej** – to klasa pochodna wskazuje na swoją klasę bazową.

```
// Klasa bazowa:
public class Person {
    // ...
}

// Klasa pochodna:
public class Employee : Person {
    // ...
}
```



- Dziedziczenie oznaczamy symbolem `:`.

Dziedziczenie składowych

Klasa **pochodna** dziedziczy pola i metody klasy **bazowej**

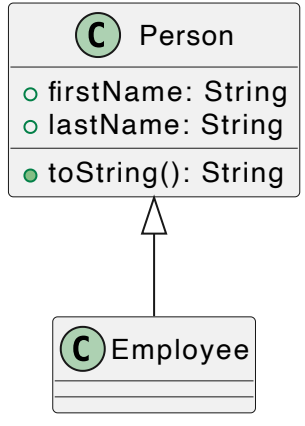
Klasa bazowa

```
public class Person {
    public String firstName;
    public String lastName;

    @Override
    public String toString() {
        return firstName + " " + lastName;
    }
}
```

Klasa pochodna

```
public class Employee extends Person {

}
```

UML



Dziedziczenie składowych

Klasa **pochodna** dziedziczy pola i metody klasy **bazowej**

Dziedziczenie nie wpływa na klasę **bazową** – działa jak poprzednio

```
Person person = new Person();
person.firstName = "Jan";
person.lastName = "Kowalski";
System.out.println(person);   // wywoła toString()
```

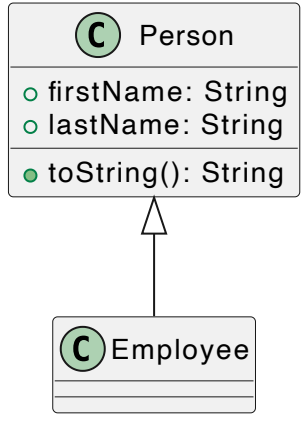
Klasa **pochodna** zyskuje składowe klasy bazowej

```
Employee employee = new Employee();
employee.firstName = "Piotr";
employee.lastName = "Nowak";
System.out.println(employee); // wywoła toString()
```

person
firstName = "Jan" lastName = "Kowalski"

employee
firstName = "Piotr" lastName = "Nowak"

UML



Dziedziczenie składowych

Klasa **pochodna** dziedziczy pola i metody klasy **bazowej**

Klasa `Employee` zachowuje się jak kopia klasy `Person`

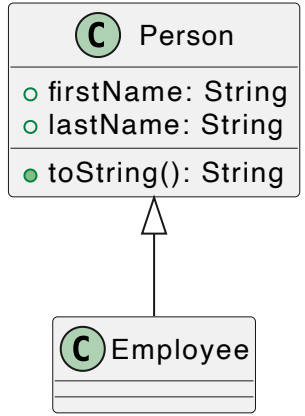
```
Employee employee = new Employee();
employee.firstName = "Piotr";
employee.lastName = "Nowak";
System.out.println(employee); // wywoła toString()
```

Efekt jest taki, jakbyśmy skopiowali całą zawartość klasy bazowej do klasy pochodnej

```
public class Employee extends Person {
    public String firstName;
    public String lastName;

    @Override
    public String toString() {
        return firstName + " " + lastName;
    }
}
```

To pozwala na ponowne wykorzystanie kodu: zamiast pisać klasę od nowa, stosujemy dziedziczenie.

UML



Rozszerzanie klasy – dodawanie pól

Klasa **pochodna** rozszerza klasę **bazową**

- Klasa pochodna może dodać własne pola.

```
public class Employee extends Person {
    public double salary;
}
```

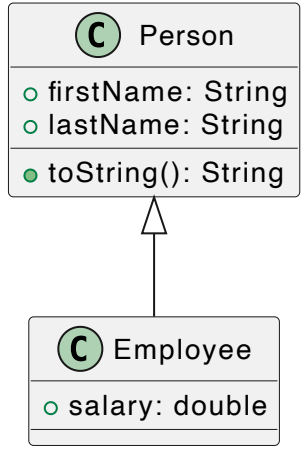
Nie ma to wpływu na klasę bazową:

```
Person person = new Person();
person.salary = 500; // ❌ Błąd, nie ma takiego pola
```

Nowe pole jest dostępne tylko w obiektach klasy `Employee`

```
Employee employee = new Employee();
employee.salary = 7500; // ✅ OK
```

UML



Rozszerzanie klasy – dodawanie metod

Klasa **pochodna** rozszerza klasę **bazową**

- Klasa pochodna może dodać własne metody.

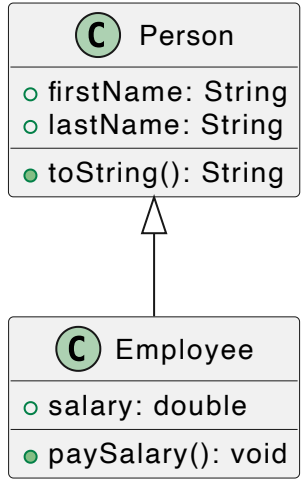
```
public class Employee extends Person {
    public double salary;
    public void paySalary() {
        System.out.println("Paying " + salary + " to " +
            firstName + " " + lastName + ".");
    }
}
```

Tak jak w wypadku pól, są one dostępne tylko w obiektach klasy `Employee`

```
Employee employee = new Employee();
employee.firstName = "Piotr";
employee.lastName = "Nowak";
employee.salary = 7500;
employee.paySalary(); // Paying 7500.0 to Piotr Nowak.
```

W metodzie `paySalary()` sięgamy do (publicznych) pól zdefiniowanych w klasie bazowej.

UML



Rozszerzanie klasy – nadpisywanie metod

Klasa **pochodna** rozszerza klasę **bazową**

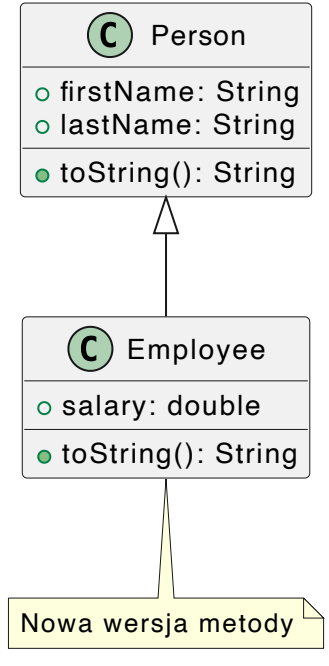
- Klasa pochodna może redefiniować metody klasy bazowej

```
public class Person {
    @Override
    public String toString() {
        return firstName + " " + lastName;
    }
}
```

Odziedziczona wersja `toString()` nam nie wystarcza, zatem dostarczamy własną:

```
public class Employee extends Person {
    @Override
    public String toString() {
        return firstName + " " + lastName +
            " (" + salary + ")";
    }
}
```

UML



Rozszerzanie klasy – nadpisywanie metod

Klasa **pochodna** nadpisuje metody klasy **bazowej**

```
public class Employee extends Person {
    @Override
    public String toString() {
        return firstName + " " + lastName +
            " (" + salary + ")";
    }
}
```

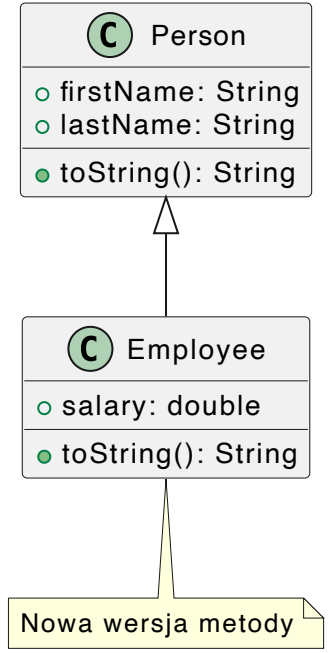
Wywołując metodę `toString()` z klasy `Employee`, wykonujemy nową, nadpisaną jej wersję.

```
Employee employee = new Employee();
employee.firstName = "Piotr";
employee.lastName = "Nowak";
employee.salary = 7500;
System.out.println(employee); // wywoła employee.toString()
```

Na ekranie pojawi się:

```
Piotr Nowak (7500.0)
```

UML



Sięganie do składowych klasy bazowej

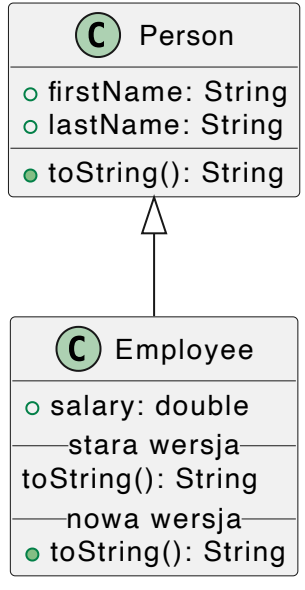
Nadpisanie metody w klasie **pochodnej** nie usuwa jej wersji **odziedziczonej** z klasy bazowej.

- Metoda klasy pochodnej przeważnie jest rozbudowaną wersją metody z klasy bazowej – wykonuje te same lub podobne działania *i coś jeszcze*.
- Aby uniknąć duplikowania kodu, w **nadpisywanej** metodzie możemy wywołać jej bazową (**odziedziczoną**) wersję:

```
public class Employee extends Person {
    @Override
    public String toString() {
        return super.toString() + " (" + salary + ")";
    }
}
```

- W **Javie** służy do tego słowo kluczowe `super`.
- Oddelegowujemy zadanie do klasy bazowej aby zachować część oryginalnego zachowania.
- W ten sposób rozszerzamy logikę, zamiast całkowicie ją zastępować.

UML



Nadpisywanie metod w Javie

Przykład: w klasie **bazowej** implementujemy metodę `save()`, która zapisuje dane np. do pliku.

```
class Person {
    public void save() {
        // Zapisujemy dane do pliku
        System.out.println("FirstName: " + firstName);
        System.out.println("LastName: " + lastName);
    }
}
```

W klasie **pochodnej** definiujemy bardziej wyspecjalizowaną wersję tej metody, która zapisze również wartość pola `salary`.

```
class Employee extends Person {
    @Override // <------------------------------ zalecane ale nieobowiązkowe
    public void save() {
        // Wywołujemy metodę z klasy bazowej
        super.save();

        // Zapisujemy dodatkowe dane
        System.out.println("Salary: " + salary);
    }
}
```

Wywołanie `super.save()` wykonuje bazowe działanie, do którego dodamy własne.

Nadpisywanie i przesłanianie metod w C++

C++ odróżnia **nadpisywanie** od **ukrywania** (*więcej na kolejnym wykładzie*).

Nadpisywanie (*overriding*) wymaga użycia słowa kluczowego `virtual`. Bez tego, mamy do czynienia z **ukrywaniem (*hiding*)**.

```
class Person {
public:
    void save() const {
        cout << "FirstName: " << firstName << endl;
        cout << "LastName: " << lastName << endl;
    }
};
```

Definiujemy nową metodę o tej samej nazwie.

```
class Employee : public Person {
public:
    void save() const {
        Person::save(); // Wywołanie metody bazowej
        cout << "Salary: " << salary << endl;
    }
};
```

Metodę z klasy bazowej możemy wywołać korzystając z operatora zakresu `::`.

Nadpisywanie i przesłanianie metod w C#

C## również odróżnia **nadpisywanie** od **ukrywania**.

```
class Person
{
    public void Save()
    {
        Console.WriteLine($"FirstName: {firstName}");
        Console.WriteLine($"LastName: {lastName}");
    }
}
```

Słowo kluczowe `new` w nagłówku metody chroni nas przed ostrzeżeniem kompilatora – to jawna deklaracja, że chcemy przesłonić metodę z klasy bazowej.

```
class Employee : Person
{
    public new void Save()
    {
        base.Save(); // Wywołanie metody z klasy bazowej
        Console.WriteLine($"Salary: {Salary}");
    }
}
```

Metodę z klasy bazowej wywołujemy korzystając z `base`.

Konstruktory w dziedziczeniu

Konstruktory (*i destruktory*) nie są dziedziczone

- Jeśli w klasie `Person` zdefiniujemy konstruktor, to czy będziemy mogli go użyć w klasie `Employee`?

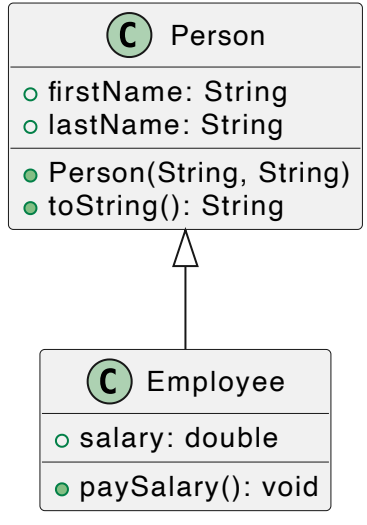
```
public class Person {
    public String firstName;
    public String lastName;

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
}
```

Niestety nie.

```
Employee employee = new Employee("Piotr", "Nowak");
//                  ^ ❌ Błąd
```

Klasa pochodna musi sama zdefiniować swoje konstruktory.

UML



Konstruktory w dziedziczeniu

Konstruktory nie są dziedziczone

- Klasa `Employee` powinna zdefiniować własny konstruktor.
- To nawet lepiej - możemy zadbać o zainicjalizowanie pola `salary`.

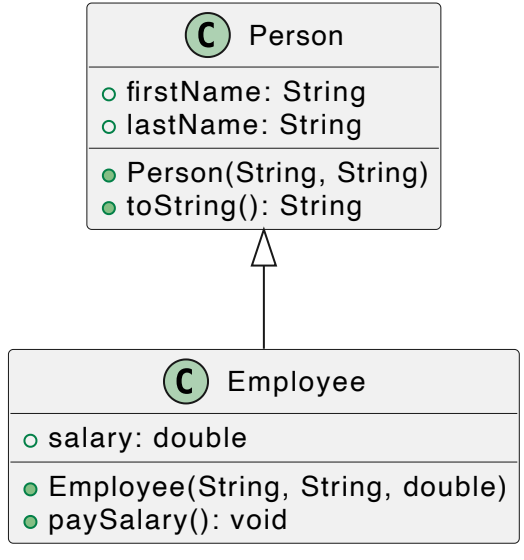
```
public class Employee extends Person {
    public double salary;

    public Employee(String firstName, String lastName,
                    double salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }
}
```

Niestety, nadal nie możemy utworzyć obiektu `Employee`.

```
Employee employee = new Employee("Piotr", "Nowak", 7500);
//                  ^ ❌ Błąd
```

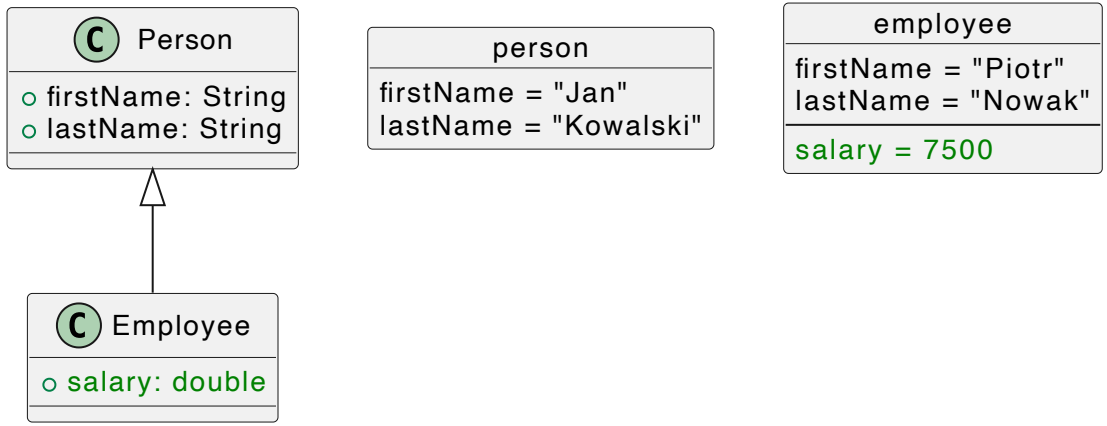
W czym leży problem?

UML



Konstruktory w dziedziczeniu

- Dlaczego konstruktor `Employee` nie działa?
- Klasa pochodna to „klasa bazowa plus dodatki” (nowe pola, nowe metody).
- Obiekt klasy pochodnej jest to „obiekt klasy bazowej z dodatkami” (z nowymi polami, nowymi metodami).
- „Wewnątrz” obiektu klasy `Employee` znajduje się obiekt klasy `Person`.
- W momencie tworzenia obiektu klasy `Employee` najpierw musi być stworzony obiekt klasy `Person`.



- A zatem musi być wywołany konstruktor klasy `Person`

Konstruktory w dziedziczeniu

- **Przed** wykonaniem konstruktora klasy **pochodnej**, wykonuje się konstruktor klasy **bazowej**.

```
public class Person {
    public Person() {
        System.out.println("Wykonuje się konstruktor klasy Person.");
    }
}
public class Employee extends Person {
    public Employee() {
        System.out.println("Wykonuje się konstruktor klasy Employee.");
    }
}
```

Po wykonaniu kodu:

```
Employee employee = new Employee();
```

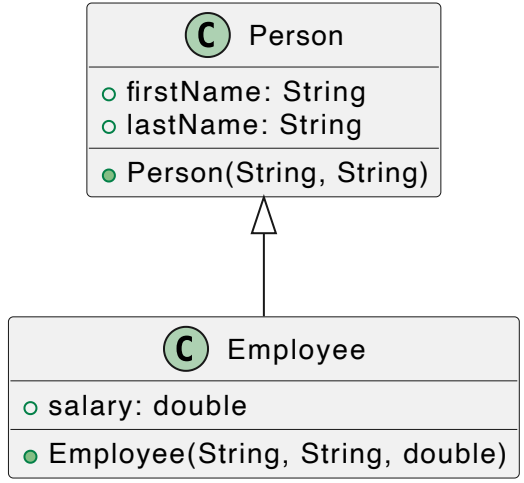
Na ekranie wyświetli się:

```
Wykonuje się konstruktor klasy Person.
Wykonuje się konstruktor klasy Employee.
```

Konstruktory w dziedziczeniu

- Dlaczego zatem konstruktor klasy `Person` się nie wykonał w tym wypadku?

```java
public class Person {
    public String firstName;
    public String lastName;

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
}

public class Employee extends Person {
    public double salary;

    public Employee(String firstName, String lastName,
                    double salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }
}
```



- **Bo go nie było** – jeśli nie określimy tego jawnie, kompilator będzie próbował wywołać konstruktor **domyślny** – czyli bezparametrowy.

Konstruktory w dziedziczeniu

- **Domyślnie** wywoływany jest kontruktor bezparametrowy klasy **bazowej**.
- Jeśli w klasie bazowej nie mamy konstruktora bezparametrowego (lub chcemy użyć innego), musimy **jawnie go wskazać** (i przekazać mu argumenty).

W **C++** robimy to w liście inicjalizacyjnej.

```
class Employee : public Person {
public:
    double salary;

    Employee(string firstName, string lastName,
             double salary): Person(firstName, lastName), salary(salary) { }
}
```

- Ponieważ **konstruktor klasy bazowej** wykona się, nie ma potrzeby powtarzania jego zadań (inicjalizowania pól `firstName` i `lastName`).
- W konstruktorze klasy `Employee` wystarczy, że zajmiemy się polami dodanymi w tej klasie.

Konstruktory w dziedziczeniu

W **Javie** służy do tego słowo kluczowe `super`.

```
public class Employee extends Person {
    public double salary;

    public Employee(String firstName, String lastName,
                    double salary) {
        super(firstName, lastName);
        this.salary = salary;
    }
}
```

- Domyślnie konstruktor klasy **pochodnej** wywołuje **domyślny** (bezparametrowy) konstruktor klasy **bazowej**.
- `super()` pozwala na wybór innego konstruktora.
- Ponadto, `super()` przekazuje konstruktorowi klasy bazowej wymagane parametry.
- Do konstruktora klasy bazowej delegujemy zadanie inicjalizacji pól odziedziczonych z klasy bazowej (`firstName` i `lastName`)
- `super()` musi być **pierwszą** instrukcją konstruktora.

Konstruktory w dziedziczeniu

W **C#** służy do tego słowo kluczowe `base`.

```
public class Employee : Person {
    public double salary;

    public Employee(string firstName, string lastName,
                    double salary): base(firstName, lastName) {
        this.salary = salary;
    }
}
```

- W celu wywołania konstruktora klasy bazowej, używa się `:base()` po parametrach konstruktora.
- Dzięki temu konstruktor `Employee` zajmuje się inicjalizacją pól zadeklarowanych w klasie `Employee`, a `Person` w klasie `Person`.

```
public class Person {
    public string firstName;
    public string lastName;

    public Person(string firstName, string lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
}
```

Destruktory w dziedziczeniu (C++)

- Destruktory są wywoływane w odwrotnej kolejności do konstruktorów: **najpierw destruktor klasy pochodnej, potem bazowej**.

```
class Person {
public:
    Person() {
        cout << "Person()" << endl;
    }
    ~Person() {
        cout << "~Person()" << endl;
    }

};

class Employee : public Person {
public:
    Employee() {
        cout << "Employee()" << endl;
    }
    ~Employee() {
        cout << "~Employee()" << endl;
    }
};
```

Po wywołaniu poniższego kodu:

```
int main()
{
    if(true) {
        // Tworzymy obiekt klasy Employee
        Employee employee;
    }
    // Obiekt jest usuwany automatycznie

    return 0;
}
```

Na ekranie wypisze się:

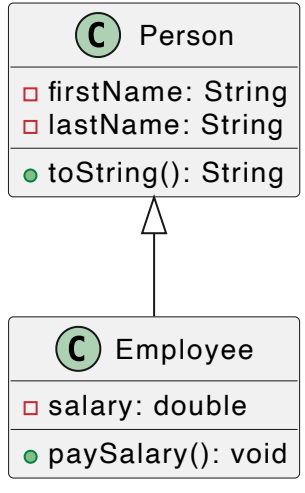
```
Person()
Employee()
~Employee()
~Person()
```

Hermetyzacja w dziedziczeniu

- **Prywatne** składowe klasy **bazowej** nie są dostępne w klasie **pochodnej**.

```
public class Person {
    private String firstName;
    private String lastName;

    @Override
    public String toString() {
        return firstName + " " + lastName;
    }
}

public class Employee extends Person {
    private double salary;
    public void paySalary() {
        System.out.println("Paying " + salary + " to " +
            firstName + " " + lastName + ".");
//          ^^^ ❌ Błąd       ^^^ ❌ Błąd
    }
}
```

UML

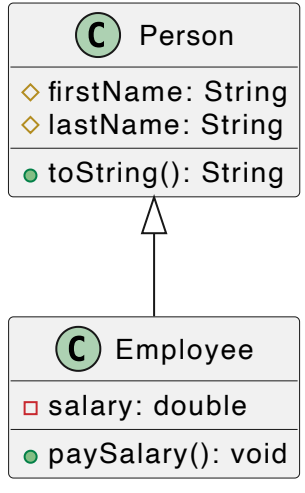


Chroniony poziom dostępu – `protected`

- **Chronione** składowe klasy bazowej **SĄ** dostępne w klasie pochodnej.

```
public class Person {
    protected String firstName;
    protected String lastName;

    @Override
    public String toString() {
        return firstName + " " + lastName;
    }
}

public class Employee extends Person {
    private double salary;
    public void paySalary() {
        System.out.println("Paying " + salary + " to " +
            firstName + " " + lastName + ".");
//          ^^^ ✅ Ok          ^^^ ✅ Ok
    }
}
```

UML



- Składowe chronione również są elementem *interfejsu klasy*.
- Uwaga: w **Javie** składowe chronione (`protected`) są publicznie dostępne dla klas tego samego pakietu.

Poziomy dziedziczenie w C++

C++ pozwala na wybór typu dziedziczenia:

Publiczne

```
class Employee : public Person { };
```

Składowe zachowują swój poziom dostępu: publiczne pozostają publiczne, chronione pozostają chronione, a prywatne pozostają prywatne.

Chronione

```
class Employee : protected Person { };
```

Publiczne składowe klasy bazowej (`Person`) stają się chronione w klasie pochodnej (`Employee`).

Są dostępne w klasie `Employee` i dla jej klas pochodnych, ale nie dla klienta.

Prywatne

```
class Employee : private Person { };
```

Publiczne i chronione składowe klasy bazowej (`Person`) stają się prywatne w klasie pochodnej (`Employee`).

Są dostępne w klasie `Employee` , ale nie dostępne dla jej klas pochodnych i kodu klienta.

Hermetyzacja w C++ - przykład

```
class Person {
public:
   string firstName;
   string lastName;
   Person(string firstName, string lastName): firstName(firstName), lastName(lastName) { }
};

class Employee: private Person { // dziedziczenie prywatne
public:
    // pola firstName i lastName stały się prywatne, ale wciąż mamy do nich dostęp
    void doWork() {
        cout << firstName << " " << lastName;
    }
protected:
    // poniższą deklaracją możemy rozszerzyć poziom dostępu
    using Person::lastName;
};

class Manager: public Employee {
public:
    void doWork() {
        cout << firstName << endl; // ❌ - pole jest prywatne w klasie bazowej
        cout << lastName << endl;  // ✅ - pole jest chronione w klasie bazowej
    }
};
```

Hermetyzacja w dziedziczeniu – praktyka

- Najlepiej unikać nadmiernego poszerzania interfejsu (również o chroniony poziom dostępu).
- Klasa powinna sama zajmować się polami, które zadeklarowała.
- Klasy pochodne, w razie potrzeby, korzystają z publicznego interfejsu.

```java
public class Person {
    private String firstName;
    private String lastName;

    public String getFirstName() {
        return firstName;
    }
    public String getLastName() {
        return lastName;
    }
    @Override
    public String toString() {
        return firstName + " " +
               lastName;
    }
}
```

```java
public class Employee extends Person {
    private double salary;

    public double getSalary() {
        return salary;
    }

    public void paySalary() {
        System.out.println(
            "Paying " +
            salary + " to " +
            getFirstName() + " " +
            getLastName() + "."
        );
    }
}
```

W `paySalary()`, zamiast metod dostępowych, można wykorzystać `toString()`:

```java
System.out.println("Paying " + salary + " to " + toString() + ".");
```

Projektowanie dziedziczenia

- To już wiemy:
  - jak działa dziedziczenie
  - co sie dziedziczy
  - jak używać `extends`, `super`, `override`, `protected`
- Ale:
  - kiedy dziedziczenie ma sens?
  - jak modelować relacje miedzy klasami?
  - kiedy dziedziczenie jest błędem?

> Czy `Firma` powinna dziedziczyć z klasy `Pracownik`?

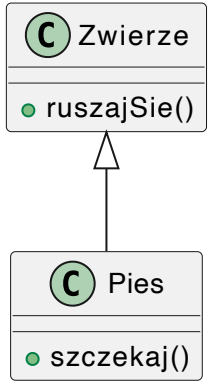
Dziedziczenie jako *specjalizacja*

Tworzymy klasę pochodną, aby uszczegółowić i rozszerzyć zachowanie klasy bazowej.

- Myślenie „od ogółu do szczegółu”
- **Punkt wyjścia: klasa bazowa** (np. `Zwierze`)
- Tworzymy nowe, bardziej konkretne typy, które **dziedziczą właściwości** i dodają nowe cechy

Przykład:

```
class Zwierze {
    void ruszajSie();
}

class Pies extends Zwierze {
    void szczekaj();
}
```



- `Pies` to **specjalizacja** `Zwierzęcia` – dodaje specyficzne zachowania.

Specjalizacja – projektowanie zaczynamy od abstrakcji, skąd wyprowadzamy konkretne przypadki.

Dziedziczenie jako *generalizacja*

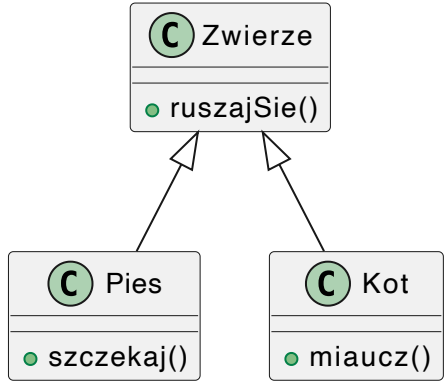
Zbieramy wspólne cechy wielu klas, by wyodrębnić ich klasę bazową.

- Myślenie „od szczegółu do ogółu”
- **Punkt wyjścia: klasy pochodne** (np. `Pies`, `Kot`)
- Tworzymy klasę bazową, by **uniknąć duplikacji** i uprościć projekt

Przykład:

```
// Mamy dwie klasy
class Pies {
    void szczekaj() { }
    void ruszajSie() { }
}

class Kot {
    void miaucz() { }
    void ruszajSie() { }
}
```

```
// Tworzymy klasę bazową
class Zwierze {
    void ruszajSie() { }
}

class Pies : Zwierze {
    void szczekaj() { }
}

class Kot : Zwierze {
    void miaucz() { }
}
```



- `Zwierze` to **generalizacja** wspólnych cech `Psa` i `Kota`.

Generalizacja – refaktoryzujemy, upraszczamy istniejący kod, szukając wspólnego mianownika.

Dziedziczenie a ponowne wykorzystanie kodu

- Dziedziczenie umożliwia ponowne użycie kodu.
- Dzięki niemu klasy pochodne mają dostęp do pól i metod, które zostały zdefiniowane w klasie bazowej.

```
public class LoggerService
{
    protected void Log(string message)
    {
        Console.WriteLine($"[{DateTime.Now}] {message}");
    }
}
```

Klasa **bazowa** dostarcza wspólną logikę. Klasa **pochodna** nie musi jej powielać, ale korzysta z odziedziczonej (*dziedziczenie implementacji*).

```
public class UserService : LoggerService
{
    public void CreateUser(string username)
    {
        // Logika tworzenia użytkownika...
        Log($"Utworzono użytkownika: {username}");
    }
}
```

Dzięki temu struktura jest bardziej przejrzysta i łatwiejsza w utrzymaniu. **Ale dziedziczenie nie powinno być używane tylko w tym celu.**

Błędne zastosowanie dziedziczenia

- Dziedziczenie w celu ponownego użycia kodu może być błędem:

```
class Point {
    double x;
    double y;
}
class Circle extends Point {
    double radius;
}
```

```
class Rectangle {
    double width;
    double height;

    public double getArea() {
        return width * height;
    }
}
class Triangle extends Rectangle {
    public double getArea() {
        return width * height * 0.5;
    }
}
```

- **Zła praktyka:** Dziedziczenie dla implementacji lub reprezentacji (dziedziczymy bo klasa „ma coś podobnego", „żeby nie pisać kodu drugi raz", bez szukania sensu semantycznego).
  - `Okrąg` nie jest specyficznym rodzajem `Punktu`, a `Trójkąt` nie jest typem `Prostokąta`
- **Dobra praktyka:** Dziedziczenie interfejsu (zachowania, kontraktu). Gdy klasa bazowa opisuje co obiekt może robić (jest generalizacją klasy pochodnej).
  - `Okrąg` zawiera `Punkt`, natomiast `Trójkąt` i `Prostokąt` są `Figurami`

Błędne zastosowanie dziedziczenia

- Dziedziczenie jest błędem, gdy prowadzi do semantycznie błędnych relacji lub gdy klasy dziedziczą niepotrzebne lub niepasujące zachowania.

```
class User
{
    string firstName;
    string lastName;
    string email;

    public void Login(string password)
    {
        // Logika logowania
    }
}
```

```
class Invoice : User
{
    double amount;

    public void Send()
    {
        // Logika wysyłania faktury
        // na wskazany adres email
    }
}
```

- `Invoice` dziedziczy z `User`, aby skorzystać z pola `email` (oraz, być może, zwiazanej z nim logiki).
- **Problemy:**
  - Faktura nie jest użytkownikiem.
  - Faktura dziedziczy też metodę `Login()`, która nie ma sensu w jej kontekście.

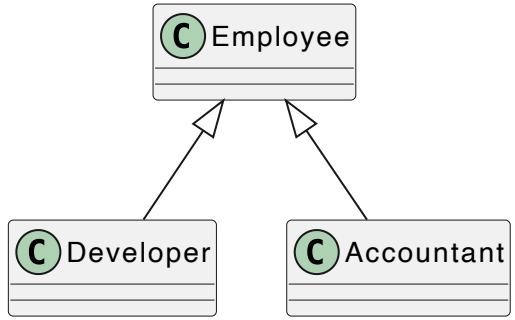
Dziedziczenie jako relacja ***is-a***

- Dziedziczenie tworzy szczególną (**bardziej specyficzną**) odmianę klasy istniejącej.
- Dziedziczenie powinno **odzwierciedlać relację semantyczną** między klasami.
- Dziedziczenie to relacja „jest rodzajem" (*is-a*) między klasą pochodną a klasą bazową

> **Klasa B dziedziczy po klasie A, jeżeli B *jest rodzajem* A.**

- To tzw. **relacja generalizacji/specjalizacji** – klasa bazowa jest ogólniejsza, a pochodna to bardziej wyspecjalizowana wersja.

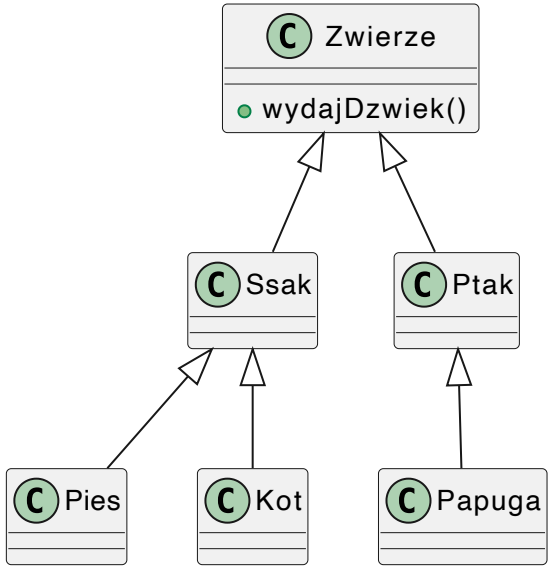
```
class Employee {
}
class Developer : Employee {
}
class Accountant : Employee {
}
```



- `Developer` i `Accountant` **są specyficznymi rodzajami** `Employee`.

Hierarchia dziedziczenia

- **Hierarchia dziedziczenia** to struktura zależności między klasami, która powstaje, gdy jedna klasa dziedziczy po innej, a ta z kolei może mieć też inne klasy potomne.
- Tworzy się w ten sposób drzewo (lub sieć) relacji typu „jest rodzajem".
- To hierarchia, bo mamy poziomy ogólności:
  - na górze: najbardziej ogólne klasy (bazowe),
  - na dole: bardziej szczegółowe klasy (pochodne).
- Taka hierarchia pozwala na reużycie kodu (klasy niżej mogą korzystać z metod klas wyżej) i lepsze modelowanie rzeczywistości (można odzwierciedlać rzeczywiste kategorie i podkategorie).



Relacja jest przechodnia (`Kot` jest `Ssakiem` a zatem `Zwierzęciem`) i jednostronna (klasa pochodna zna bazową).

Dziedziczenie typu

- Klasa dziedziczy nie tylko składowe ale również typ.

```
class Employee : Person {
    public bool LogIn(string password) { }
}

class Accountant : Employee {
    // dziedziczymy metodę LogIn()
}

class MailService {
    public static void SendMessage(Employee employee, string message) {
        // Logika wysyłania wiadomości do pracownika
    }
}

Accountant accountant = new Accountant("Jan", "Kowalski", "janko@company.com")
// Dozwolone, ponieważ księgowy jest pracownikiem
MailService.SendMessage(accountant, "Wesołych świąt dla załogi!")
```

- Dziedziczenie składowych to najbardziej „namacalne” dziedziczenie.
- Dziedziczenie typu jest bardziej semantyczne: **Klasa** `Accountant` **jest traktowana jako typ** `Employee`, czyli można jej używać wszędzie tam, gdzie oczekiwany jest `Employee`.
  - `Accountant` dziedziczy tożsamość typową po `Employee`

Podsumowując

Po co dziedziczenie?

1. Ujednolicenie interfejsu: zarówno `FileLogger`, `DbLogger` jak i `ConsoleLogger` to `Logger`, każdy pozwala na to samo.
2. Rozszerzanie zachowania: `EncryptedFileWriter` to `FileWriter` rozbudowany o szyfrowanie.
3. Modelowanie rzeczywistości: `Karetka` jest `Pojazdem` a zatem ma wszystkie cechy `Pojazdu`.

Kiedy używać dziedziczenia?

1. Gdy istnieje relacja ***is-a***, np. `Kot` to `Zwierzę`.
2. Gdy chcemy rozszerzyć istniejące zachowanie, np. `Kot` potrafi to, co każde `Zwierzę` (i coś więcej).
3. Gdy chcemy różne typy obiektów traktować jednakowo, np. zarówno `Kot`, `Pies` i `Ptak` to `Zwierzę`.

Kiedy **nie** używać?

1. Tylko po to „by nie powielać kodu", np. `Pracownik` dziedziczy po `Dokument` bo chcemy skorzystać z jego gotowej metody `zapiszDoPliku()`.
2. Gdy relacja nie pasuje, np. jest pewien logiczny związek między `Samolot` a `Samochód` albo `Silnik` i `Samochód` ale żadne nie jest specyficznym typem drugiego.
3. Gdy zaczynamy dziedziczyć niepasujące zachowania, np. `Lokata` może wydawać się specyficznym rodzajem `Konta` ale dziedzicząc otrzymujemy operacje, które nie powinny być dostępne.